Geth 가비지 컬렉션 전략 분석

개념적 배경

왜 가비지 컬렉션이 필요한가?

Ethereum 노드는 지속적으로 증가하는 상태 데이터를 관리해야 합니다:

1. **메모리 제약**: 무제한 메모리 사용은 시스템 안정성을 위협
2. **상태 트리 변화**: 블록마다 변경되는 상태로 인한 오래된 노드 축적
3. **참조 관계 복잡성**: 트라이 노드 간의 복잡한 참조 관계 관리 필요
4. **디스크 공간 최적화**: 불필요한 데이터의 디스크 저장 방지

Geth의 가비지 컬렉션 접근법

Geth는 두 가지 주요 스키마에서 서로 다른 GC 전략을 사용합니다:

1. **HashDB**: 참조 카운팅 기반 가비지 컬렉션
2. **PathDB**: 계층적 diff 관리와 주기적 압축

구현 분석

1. HashDB 가비지 컬렉션 (`triedb/hashdb/`)

참조 카운팅 시스템

```
type cachedNode struct {
node []byte  // 노드 데이터
```

```
parents  uint32  // 참조 카운트
external  map[common.Hash]struct{} // 외부 참조 집합
flushPrev common.Hash // 플러시 리스트 이전 노드
flushNext common.Hash // 플러시 리스트 다음 노드
}
```

참조 추가 메커니즘

```
func (db *Database) reference(child common.Hash, parent common.Hash) {
node, ok := db.dirties[child]
if !ok {
return  // 디스크에서 로드된 노드는 스킵
}
// 루트 상태에 대한 참조인 경우
if  parent == (common.Hash{}) {
node.parents += 1
return
}
// 외부 스토리지 트라이 참조 처리
if  db.dirties[parent].external == nil {
db.dirties[parent].external = make(map[common.Hash]struct{})
}
// 중복 참조 방지
```

```
if _, ok := db.dirties[parent].external[child]; ok {
return
}
node.parents++
db.dirties[parent].external[child] = struct{}{}
db.childrenSize += common.HashLength
}
```

참조 해제 및 가비지 컬렉션

```
func (db *Database) dereference(hash common.Hash) {
node, ok := db.dirties[hash]
if !ok {
return // 이미 커밋된 노드
}
// 참조 카운트 감소
if node.parents > 0 {
node.parents--
}
// 참조가 없으면 제거
if node.parents == 0 {
// 플러시 리스트에서 제거
db.removeFromFlushList(hash, node)
// 모든 자식 노드 재귀적 해제
```

```
node.forChildren(func(child common.Hash) {
db.dereference(child)
})
// 노드 삭제 및 메모리 해제
delete(db.dirties, hash)
db.dirtiesSize -= common.StorageSize(common.HashLength + len(node.node))
if  node.external != nil {
db.childrenSize -= common.StorageSize(len(node.external) * common.HashLength)
}
}
}
```

메모리 압박 시 캐시 플러시

```
func (db *Database) Cap(limit common.StorageSize) error {
db.lock.Lock()
defer  db.lock.Unlock()
batch := db.diskdb.NewBatch()
nodes, storage, start := len(db.dirties), db.dirtiesSize, time.Now()
// 현재 메모리 사용량 계산
size := db.dirtiesSize + common.StorageSize(len(db.dirties)*cachedNodeSize)
size += db.childrenSize
// 제한을 초과하는 동안 가장 오래된 노드부터 플러시
```

```
oldest := db.oldest

for  size > limit && oldest != (common.Hash{}) {

node := db.dirties[oldest]

// 디스크에 쓰기

rawdb.WriteLegacyTrieNode(batch, oldest, node.node)

// 배치 크기 확인 및 커밋

if  batch.ValueSize() >= ethdb.IdealBatchSize {

if  err := batch.Write();  err != nil {

return  err

}

batch.Reset()

}

// 메모리 사용량 업데이트

size -= common.StorageSize(common.HashLength + len(node.node) + cachedNodeSize)

if  node.external != nil {

size -= common.StorageSize(len(node.external) * common.HashLength)

}

oldest = node.flushNext

}

// 남은 배치 데이터 커밋

if  err := batch.Write();  err != nil {

return  err

}
```

```
// 플러시된 노드들을 메모리에서 제거

db.cleanupFlushedNodes(oldest)

return nil

}
```

2. PathDB 가비지 컬렉션 (`triedb/pathdb/`)

계층적 Diff 관리

```
const (

maxDiffLayers = 128  // 최대 diff 레이어 수

defaultBufferSize = 64 * 1024 * 1024  // 기본 버퍼 크기

maxBufferSize = 256 * 1024 * 1024  // 최대 버퍼 크기

)


type Config struct {

StateHistory  uint64  // 유지할 최근 블록 수

EnableStateIndexing  bool  // 상태 히스토리 인덱싱 활성화

TrieCleanSize  int  // 클린 트라이 노드 캐시 크기

StateCleanSize  int  // 클린 상태 데이터 캐시 크기

WriteBufferSize  int  // 쓰기 버퍼 크기

}
```

Diff Layer 압축

```go
func (dl *diffLayer) flatten() *diffLayer {
// 부모 레이어와 현재 레이어 병합
parent := dl.parent.(*diffLayer)
// 노드 집합 병합
nodes := make(map[string]map[common.Hash]*trienode.Node)
for owner, set := range parent.nodes {
nodes[owner] = make(map[common.Hash]*trienode.Node)
for path, node := range set {
nodes[owner][path] = node
}
}
// 현재 레이어의 변경사항 적용
for owner, set := range dl.nodes {
if nodes[owner] == nil {
nodes[owner] = make(map[common.Hash]*trienode.Node)
}
for path, node := range set {
nodes[owner][path] = node
}
}
// 상태 변경사항 병합
states := dl.states.copy()
```

```
states.merge(parent.states)

return &diffLayer{

root: dl.root,

id: dl.id,

block: dl.block,

nodes: nodes,

states: states,

parent: parent.parent,

memory: dl.memory + parent.memory,

}

}
```

3. 통합 가비지 컬렉션 제어

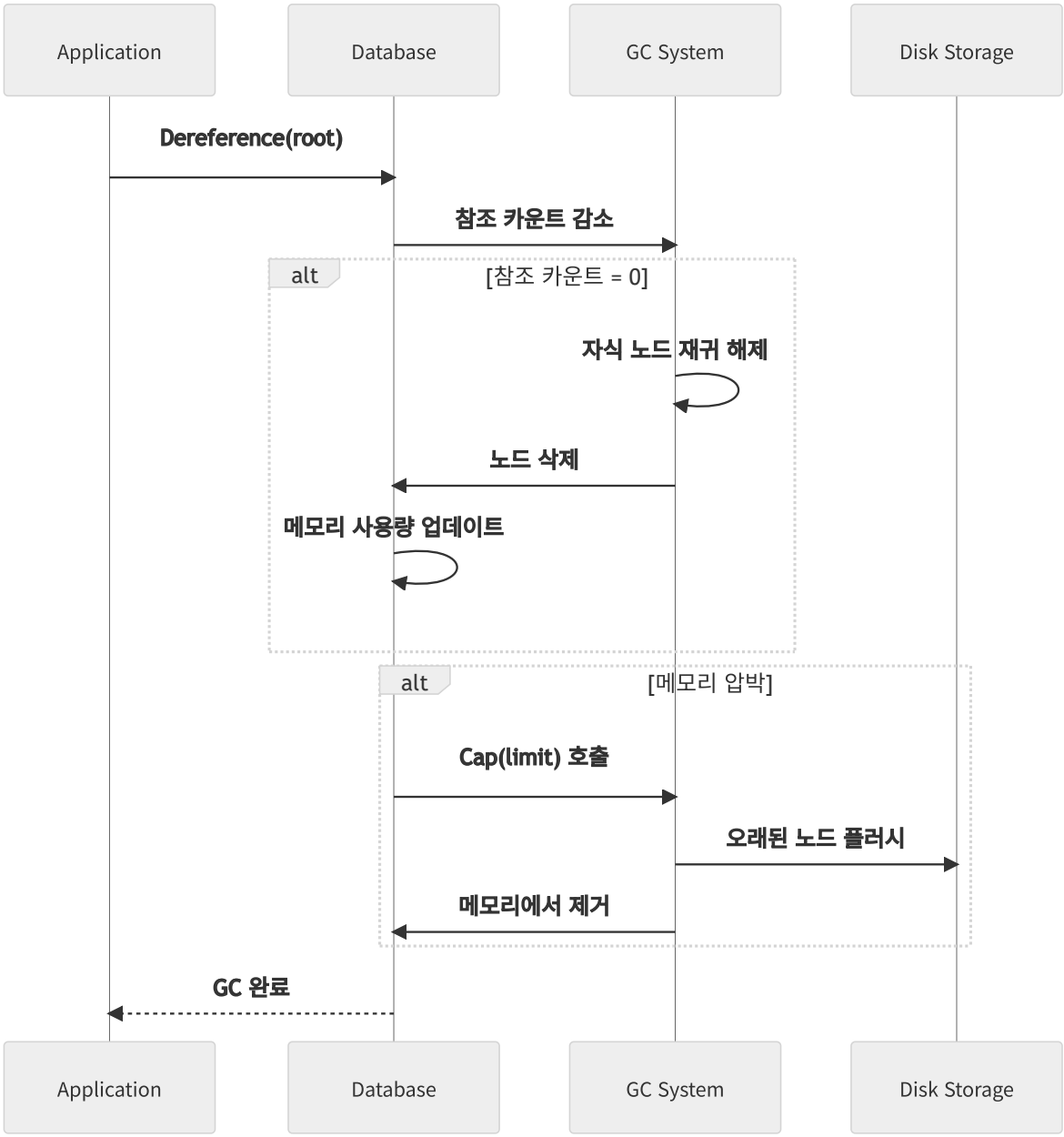
자동 가비지 컬렉션 트리거

```
func (db *Database) Dereference(root common.Hash) {

if  root == (common.Hash{}) {

log.Error("Attempted to dereference the trie cache meta root")

return

}

db.lock.Lock()

defer  db.lock.Unlock()

nodes, storage, start := len(db.dirties), db.dirtiesSize, time.Now()
```

```
db.dereference(root)

// GC 통계 업데이트

db.gcnodes += uint64(nodes - len(db.dirties))

db.gcsize += storage - db.dirtiesSize

db.gctime += time.Since(start)

// 메트릭 업데이트

memcacheGCTimeTimer.Update(time.Since(start))

memcacheGCBytesMeter.Mark(int64(storage - db.dirtiesSize))

memcacheGCNodesMeter.Mark(int64(nodes - len(db.dirties)))

}
```

실행 흐름

가비지 컬렉션 생명주기



메모리 압박 상황 처리

```
func (bc *BlockChain) maintainTxIndex(ancients  uint64) {

// 주기적으로 실행되는 유지보수 작업
```

```
defer bc.wg.Done()

// 트라이 캐시 크기 모니터링

for {

select {

case <-bc.quit:

return

case <-time.After(time.Minute):

// 메모리 사용량 확인

if bc.triedb.Size() > bc.cacheConfig.TrieDirtyLimit {

// 강제 가비지 컬렉션 실행

bc.triedb.Cap(bc.cacheConfig.TrieDirtyLimit)

}

}

}

}
```

성능 최적화

1. 참조 카운팅 최적화

지연 삭제 (Lazy Deletion)

```
func (db *Database) scheduleGC(hash common.Hash) {

// 즉시 삭제하지 않고 GC 스케줄에 추가
```

```
db.gcQueue = append(db.gcQueue, hash)

// 배치 크기에 도달하면 일괄 처리

if len(db.gcQueue) >= gcBatchSize {

db.processBatchGC()

}

}
```

참조 카운트 캐싱

```
type refCountCache struct {

counts map[common.Hash]uint32

mutex sync.RWMutex

}


func (rc *refCountCache) increment(hash common.Hash) {

rc.mutex.Lock()

defer rc.mutex.Unlock()

rc.counts[hash]++

}
```

2. 메모리 관리 최적화

적응적 캐시 크기 조정

```
func (db *Database) adaptiveCacheSize() common.StorageSize {
```

```
// 시스템 메모리 사용률에 따른 동적 조정

memStats := &runtime.MemStats{}

runtime.ReadMemStats(memStats)

availableMemory := memStats.Sys - memStats.Alloc

// 사용 가능한 메모리의 일정 비율을 캐시에 할당

return  common.StorageSize(availableMemory * cacheMemoryRatio)

}
```

메모리 풀링

```
var  nodePool = sync.Pool{

New: func() interface{} {

return &cachedNode{

external: make(map[common.Hash]struct{}),

}

},

}


func (db *Database) allocateNode() *cachedNode {

return  nodePool.Get().(*cachedNode)

}


func (db *Database) releaseNode(node *cachedNode) {

// 노드 초기화
```

```
node.node = nil
node.parents = 0
clear(node.external)
nodePool.Put(node)
}
```

3. 디스크 I/O 최적화

배치 쓰기 최적화

```
func (db *Database) flushBatch(batch ethdb.Batch, nodes []*cachedNode) error {
// 노드들을 크기 순으로 정렬하여 디스크 지역성 향상
sort.Slice(nodes, func(i, j int) bool {
return len(nodes[i].node) < len(nodes[j].node)
})
for _, node := range nodes {
batch.Put(node.hash[:], node.node)
// 배치 크기 확인
if batch.ValueSize() >= ethdb.IdealBatchSize {
if err := batch.Write(); err != nil {
return err
}
batch.Reset()
}
```

```
}
return batch.Write()
}
```

실제 사용 예시

1. 블록 커밋 시 가비지 컬렉션

```
func (bc *BlockChain) writeBlockWithState(block *types.Block, receipts []*types.
logs []*types.Log, state *state.StateDB) error {
// 상태 커밋
root, err := state.Commit(bc.chainConfig.IsEIP158(block.Number()))
if err != nil {
return err
}
// 트라이 데이터베이스 커밋
if err := bc.triedb.Commit(root, true); err != nil {
return err
}
// 이전 상태 참조 해제 (가비지 컬렉션 트리거)
if parent := bc.GetBlock(block.ParentHash(), block.NumberU64()-1); parent != n
bc.triedb.Dereference(parent.Root())
}
```

```
return nil
}
```

2. 메모리 압박 시 자동 정리

```
func (bc *BlockChain) maintainCaches() {
ticker := time.NewTicker(5 * time.Minute)
defer ticker.Stop()
for {
select {
case <-ticker.C:
// 메모리 사용량 확인
_, dirty := bc.triedb.Size()
if dirty > bc.cacheConfig.TrieDirtyLimit {
log.Info("Triggering trie cache cleanup",
"dirty", dirty, "limit", bc.cacheConfig.TrieDirtyLimit)
// 강제 캐시 정리
if err := bc.triedb.Cap(bc.cacheConfig.TrieDirtyLimit / 2); err != nil {
log.Error("Failed to cap trie cache", "err", err)
}
}
case <-bc.quit:
return
```

```
}
}
}
```

3. 설정 기반 가비지 컬렉션 튜닝

```
type CacheConfig struct {
TrieCleanLimit  int  // 클린 캐시 크기 (MB)
TrieDirtyLimit  int  // 더티 캐시 크기 (MB)
TrieTimeLimit time.Duration // 플러시 시간 제한
SnapshotLimit  int  // 스냅샷 캐시 크기 (MB)
SnapshotWait  bool  // 스냅샷 대기 여부
}

func  DefaultCacheConfig() *CacheConfig {
return &CacheConfig{
TrieCleanLimit: 256,
TrieDirtyLimit: 256,
TrieTimeLimit: 5 * time.Minute,
SnapshotLimit: 256,
SnapshotWait: true,
}
```

```
}
```

관련 시스템

의존성 관계

1. **상태 관리 시스템**: 상태 변경 시 GC 트리거
2. **블록체인 동기화**: 블록 커밋 시 이전 상태 정리
3. **메모리 관리**: 시스템 메모리 압박 시 자동 정리
4. **성능 모니터링**: GC 성능 메트릭 수집

상호작용 패턴

- **주기적 정리**: 시간 기반 자동 가비지 컬렉션
- **압박 기반 정리**: 메모리 사용량 임계값 도달 시 강제 정리
- **참조 기반 정리**: 참조 카운트 0 도달 시 즉시 정리

영향 범위

가비지 컬렉션 성능은 다음 영역에 영향을 미칩니다:

- **메모리 사용량**: 효율적인 메모리 관리로 안정성 향상
- **디스크 I/O**: 배치 쓰기로 디스크 성능 최적화
- **응답 시간**: GC 일시정지 시간 최소화로 응답성 향상
- **시스템 안정성**: 메모리 누수 방지로 장기 실행 안정성 확보

이러한 포괄적인 가비지 컬렉션 전략을 통해 Geth는 메모리 효율성과 성능을 동시에 달성하며, 장기간 안정적인 노드 운영을 보장합니다.